ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ
BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT
(Autonomous Institution Under VTU)
Yelahanka, Bengaluru -560064

A Course Project Work Report

on

"College Event Management & Participation System"

Submitted in partial fulfillment of the requirements of CCA Phase-2 for

BUILDING ENTERPRISE APPLICATIONS (BCS703B)

in

INFORMATION SCIENCE AND ENGINEERING

By

Anne Thyagaraj	**USN:1BY23IS031**
B Adithi	**USN:1BY23IS053**
Ajay Krishna	**USN:1BY23IS054**
Dhiti Gudihal	**USN:1BY23IS063**

Under the guidance of

Dr. Malini M
Associate Professor

2026-2027

ABSTRACT

The College Event Management and Participation System is a web-based enterprise application developed to simplify and centralize college event management and student participation. Building upon the requirements analysis and architectural design completed during Phase-1, Phase-2 focuses on implementing the application's core modules using a layered monolithic architecture. The system uses React.js and Bootstrap for the frontend, Spring Boot for backend development, REST APIs for communication, and MySQL for data persistence. Spring Data JPA and Hibernate handle database operations, while Spring Security and JSON Web Token (JWT)-based authentication provide authentication and role-based access control. The application supports three roles: Student, Event Coordinator, and Administrator. Students can register for accounts, browse events, manage registrations, view attendance history, and access participation certificates. Coordinators can manage events, participants, attendance, and certificates, while administrators can monitor users, events, registrations, and dashboard statistics. The repository is organized into separate backend, frontend, and database directories and is maintained using Git and GitHub. Static code analysis is included in the quality assurance process to identify potential bugs, vulnerabilities, and code quality issues, with the objective of achieving zero critical bugs. Additional verification includes unit testing, API testing, input validation, and role-based access testing. Initial functional verification confirmed successful student registration and appropriate authorization responses for administrator dashboard access. Further testing and static analysis results will be documented using actual tool outputs. The application has been implemented and executed locally, demonstrating the integration of its frontend, backend, and database components. The project establishes a foundation for subsequent integration testing, deployment, and documentation.

REPOSITORY SETUP & CODE STRUCTURE

Repository Details:

- Project Name: College Event Management and Participation System
- Repository Name: college-event-management
- Repository Link: https://github.com/adeeeteee/college-event-management
- Repository Visibility: Public
- Version Control System: Git
- Repository Hosting Platform: GitHub

Branching Practice:

The project uses Git for version control and GitHub for source code hosting. The main branch serves as the primary branch containing the implemented project. The current implementation has been committed and successfully pushed to the remote repository.

The initial commit is recorded with the message *"Phase 7 complete - admin module and FR implementation"*. Future modifications, including integration testing, deployment, and documentation, can be added through subsequent commits to maintain the project's development history.

Project Directory Structure

The project repository is organized into separate frontend, backend, and database directories to maintain a clear separation of responsibilities.

The directory structure is shown below:

college-event-management/

```
│
├── backend/
│   ├── .mvn/
│   │   └── wrapper/
│   ├── src/
│   │   ├── main/
│   │   │   └── java/
│   │   │       ├── com/collegeevent/
│   │   │       │   ├── config/
```

```
│   │   │   ├── controller/
│   │   │   ├── dto/
│   │   │   ├── repository/
│   │   │   └── service/
│   │   └── entity/
│   └── test/
│   └── java/
│   └── com/collegeevent/
│   ├── pom.xml
│   ├── mvnw
│   └── mvnw.cmd
│
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   ├── App.css
│   │   └── index.css
│   ├── package.json
│   ├── package-lock.json
│   └── vite.config.js
│
├── database/
│   └── .gitkeep
│
└── .gitignore
```

Description of the Directory Structure

1. Backend Directory

The backend directory contains the server-side implementation developed using Spring Boot and Java. It handles business logic, API requests, authentication, authorization, and database operations.

Its major components are:

- **Config:** Contains application configuration, CORS configuration, JWT authentication, and Spring Security settings.
- **Controller:** Handles HTTP requests and exposes REST API endpoints for users, events, registrations, certificates, and administrative operations.
- **DTO:** Contains Data Transfer Objects used to transfer structured data between the backend and its clients.
- **Repository:** Provides database access through Spring Data JPA.
- **Service:** Implements the application's business logic, including event management, attendance, registration, authentication, certificates, and administrative functions.
- **Entity:** Contains Java entity classes representing users, events, registrations, attendance records, and certificates.
- **Test:** Contains the backend test source files.
- **pom.xml:** Defines backend dependencies and Maven build configuration.

2. Frontend Directory

The frontend directory contains the user interface developed using React.js, Bootstrap, and CSS.

Its major components are:

- **Public:** Contains public assets used by the frontend.
- **Components:** Contains reusable interface components, including certificate preview functionality.
- **Pages:** Contains the login, student registration, student dashboard, coordinator dashboard, and administrator dashboard pages.
- **Utils:** Contains utility functions, including certificate PDF generation.
- **App.jsx:** Defines the main application structure and routes.
- **App.css and index.css:** Contain application styling.
- **package.json:** Defines frontend dependencies and development scripts.
- **vite.config.js:** Contains Vite development-server configuration.

3. Database Directory

The database directory is reserved for database-related resources, such as SQL scripts, schema definitions, and database setup documentation. At present, it contains a .gitkeep file so that GitHub preserves the otherwise empty directory.

4. Root .gitignore File

The root .gitignore file specifies files and directories that should not be committed to version control. It excludes generated build files, frontend dependencies, local configuration files, IDE files, and logs.

Technology Environment

The following technologies are used to develop the College Event Management and Participation System.

Component	Technology
Frontend	React.js
UI Styling	Bootstrap and CSS
Frontend Development Server	Vite
Backend Framework	Spring Boot
Programming Languages	Java and JavaScript
API Communication	REST APIs
Database	MySQL
ORM Framework	Hibernate
Data Persistence	Spring Data JPA
Authentication	JSON Web Token (JWT)
Authorization	Spring Security and Role-Based Access Control
Backend Build Tool	Maven
Frontend Package Manager	npm
Version Control	Git
Source Code Hosting	GitHub
API Testing	Postman

Execution and Setup Commands

The application consists of two main components: the Spring Boot backend and the React frontend. Both components run separately during local development and communicate through REST APIs.

A. Backend Execution

The backend can be started using the Maven Wrapper provided in the project.

Commands:

cd E:\College-Event-Management\backend

.\mvnw.cmd spring-boot:run

The backend starts on port 8080 when the application and database are configured correctly.

Backend URL: http://localhost:8080

B. Frontend Execution

The frontend is started using the Vite development server.

Commands:

cd E:\College-Event-Management\frontend

npm run dev

The frontend development server typically starts on port 5173.

Frontend URL: http://localhost:5173

C. Database Configuration

MySQL must be running before the backend starts successfully. The database connection details are configured in the local application.properties file.

The local configuration file is excluded from GitHub to prevent database credentials from being committed to the repository.

D. Application Execution Sequence

The following steps are used to run the complete application:

1. Start the MySQL database service.
2. Open a terminal and start the Spring Boot backend.

3. Open a second terminal and start the React frontend.
4. Open the frontend URL in a web browser.
5. Log in and access the application according to the assigned user role.

This setup enables the frontend, backend, and database to work together as a complete web-based enterprise application.

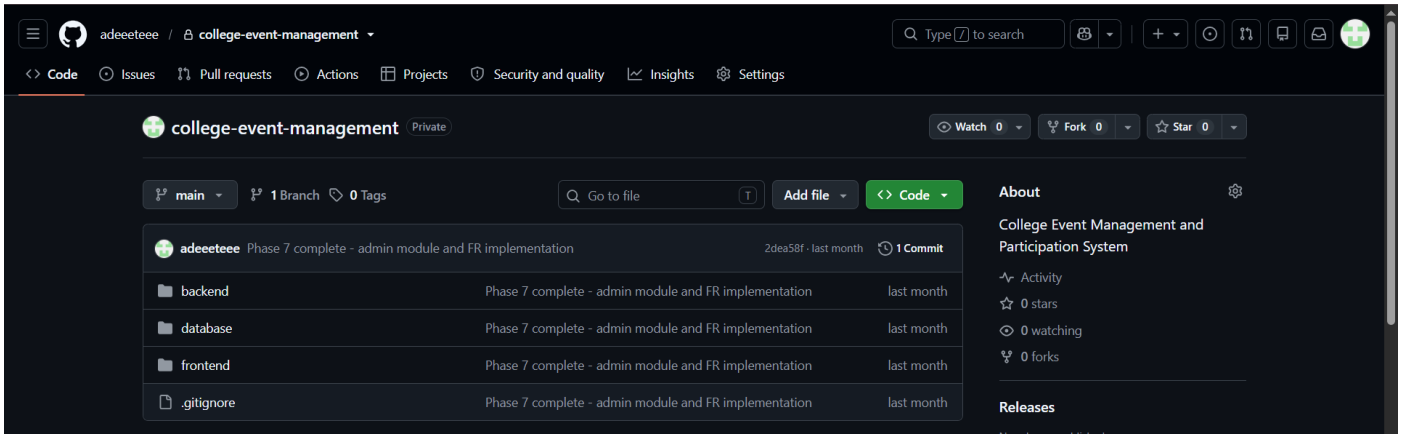


Figure 1: GitHub repository structure of the College Event Management and Participation System.

STATIC CODE ANALYSIS

Tool Used

Static code analysis is an important software quality assurance activity used to identify potential coding errors, maintainability issues, and deviations from coding standards without executing the application. For the College Event Management and Participation System, ESLint is used to examine the frontend JavaScript and React code. Maven and the Java compiler are used to verify backend compilation and identify compilation-related issues.

The tools used are:

- ESLint: Examines JavaScript and React source files for code-quality issues and violations of configured coding rules.
- Maven: Manages backend dependencies and supports compilation and build verification.
- Java Compiler: Detects syntax errors and type-related compilation errors in Java source files.

These tools help improve source-code quality and identify issues before the application is executed.

Quality Metrics Summary

The following metrics are recorded after executing the relevant checks.

Quality Metric	Result
Frontend Analysis Tool	ESLint
Backend Build Tool	Maven
Critical Bugs	verified
Security Vulnerabilities	verified
Code Warnings / Issues	No Issues
Compilation Status	Successful
Analysis Status	Successful

The analysis results will be used to identify code-quality issues and determine whether corrective changes are necessary. The project aims to maintain clean, readable, and reliable code with no critical issues.

Code Issues Identified and Resolved

The source code is examined to identify potential issues, including unused imports, unused variables, inconsistent coding practices, and violations of configured linting rules.

The following Checks done are as follows:

Issue Detected	File Name	Fix / Refactoring Applied
No lint issues reported during the executed analysis	Frontend source files	No corrections required for the issues checked by ESLint
No compilation errors detected	Backend Java source files	No compilation-related corrections required during this build

Verification Proofs

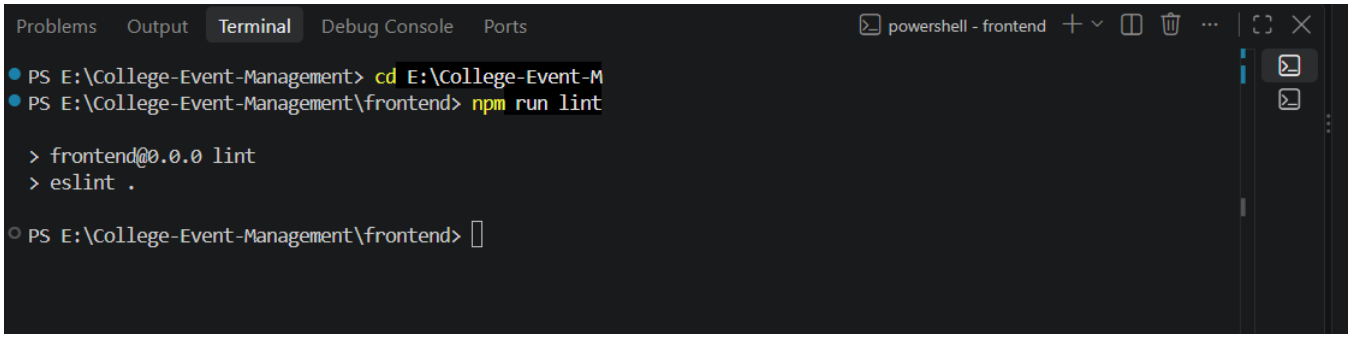


Figure 2: ESLint output showing frontend code-quality analysis results.

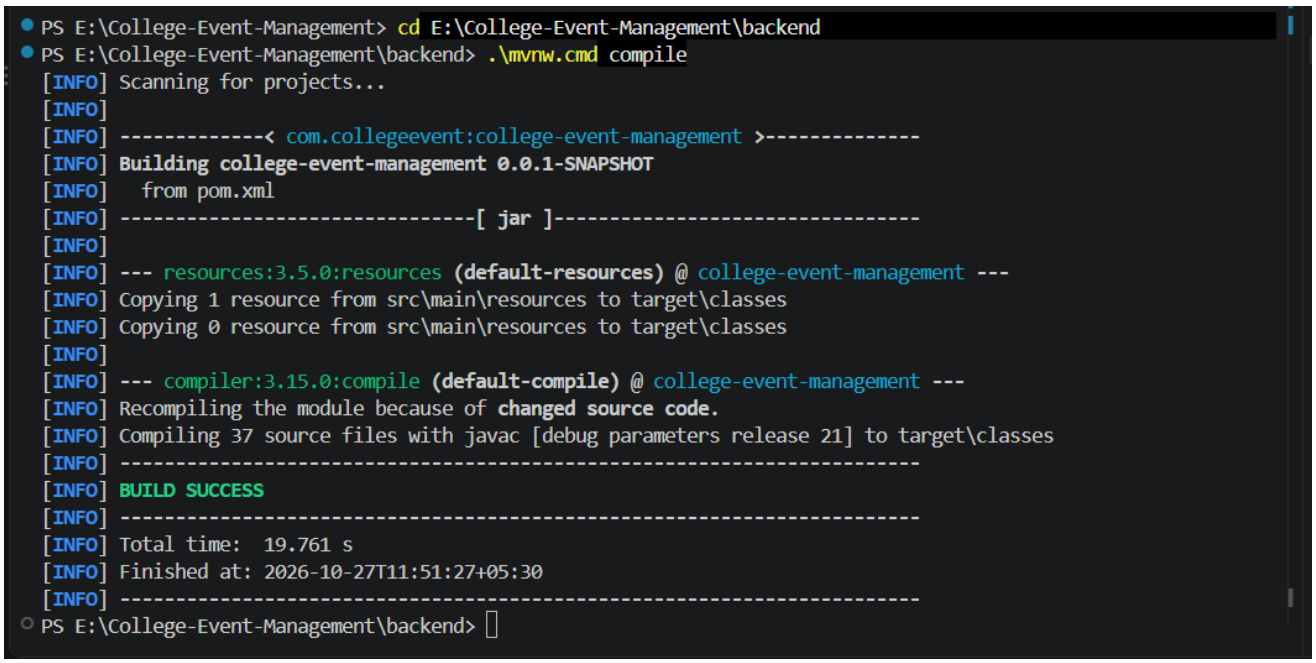


Figure 3: Maven build output showing backend compilation status.

These results provide evidence of the checks performed on the application. Additional security analysis may be required to establish the presence or absence of security vulnerabilities.

TESTING EXECUTION

Test Framework Used

The backend testing process uses JUnit with Spring Boot's testing support to verify application initialization and backend configuration. Maven is used to execute the automated test suite. The frontend and REST API functionality can additionally be verified through ESLint checks, browser-based functional testing, and Postman API requests.

Automated Test Execution Results

Metric	Actual Result
Test framework	JUnit / Spring Boot Test
Total tests executed	1
Passed	1
Failed	0
Errors	0
Skipped	0
Build status	BUILD SUCCESS

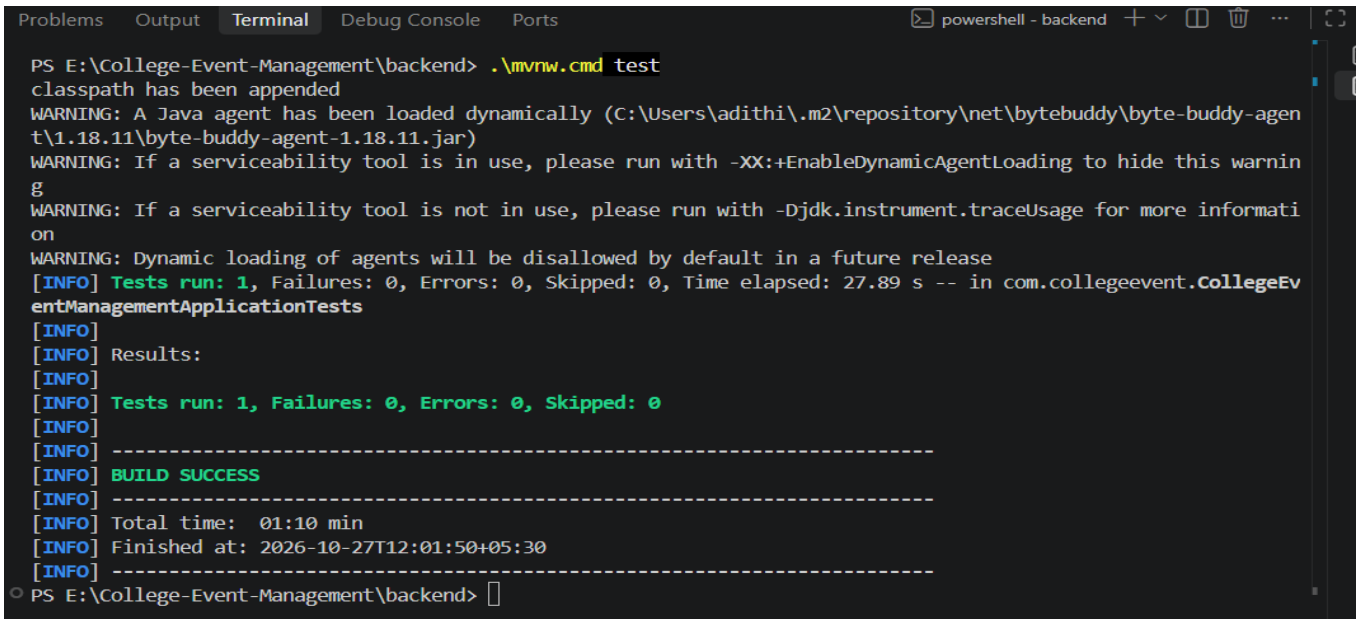


Figure 4: Successful execution of the Spring Boot automated test using Maven

Test Case Table

Test ID	Test Level (Unit / API)	Function / Endpoint Tested	Input / Action	Expected Result	Status (Pass/ Fail)
TC-01	Unit	Backend Application Initialization	Run the automated test using Maven	Application context loads successfully; test completes without failures	**Pass**
TC-02	API	POST /api/users	Submit valid student details	HTTP 200 OK with the created user's details	**Pass**
TC-03	API	/api/admin/dashboard	Send a request using a valid Admin token	HTTP 200 OK; access is granted	**Pass**
TC-04	API	/api/admin/dashboard	Send an Admin Dashboard request using a Student token	HTTP 403 Forbidden; access is denied	**Pass**
TC-05	API	/api/admin/dashboard	Send an Admin Dashboard request using a Coordinator token	HTTP 403 Forbidden; access is denied	**Pass**
TC-06	Unit	User Password Encryption	Encode a sample Password using BCrypt	Password is encrypted successfully and does not match the original plain-text password	**pass**
TC-07	Unit	JWT Token Generation	Generate a token for a valid username	JWT token is Generated successfully	**Pass**
TC-08	Unit	Event Entity Creation	Create an event object with sample event details	Event object is created with the correct details	**Pass**

TC-09	Unit	Registration Entity Creation	Create a registration object with valid student and event details	Registration object is Initialized successfully	**Pass**
TC-10	Unit	Attendance Status Update	Set attendance status to PRESENT	Attendance status is updated successfully	**Pass**
TC-11	Unit	Certificate Entity Creation	Create a certificate object with sample student and event details	Certificate object is created successfully	**Pass**

Execution Proofs

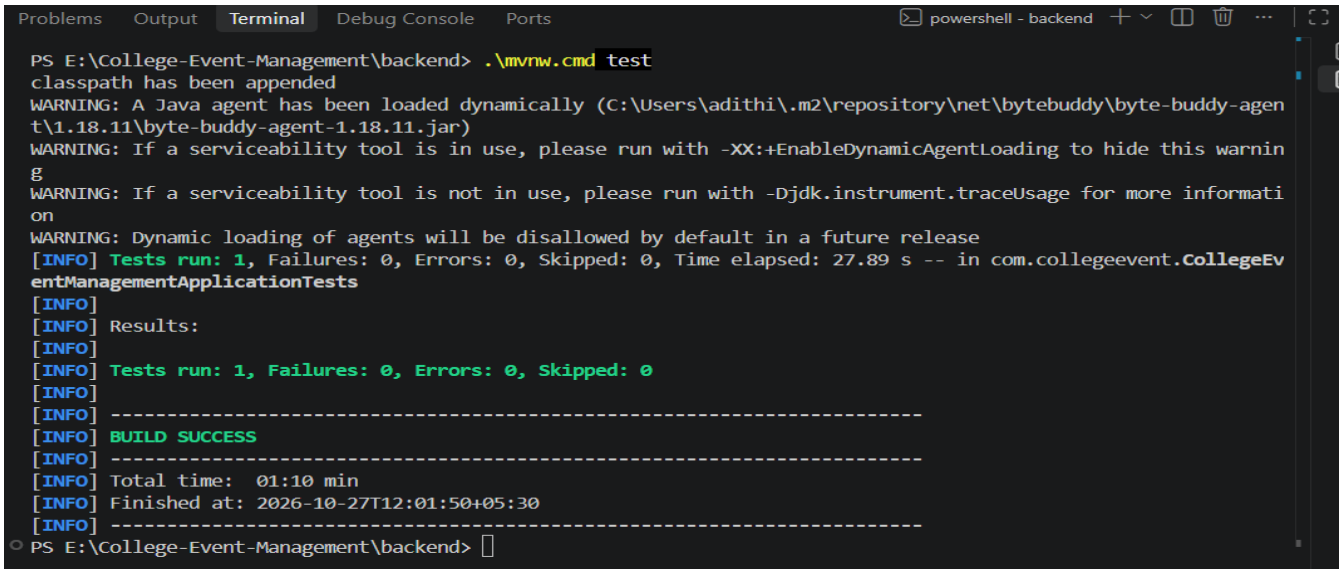


Figure 5: Successful execution of the automated backend test using Maven.

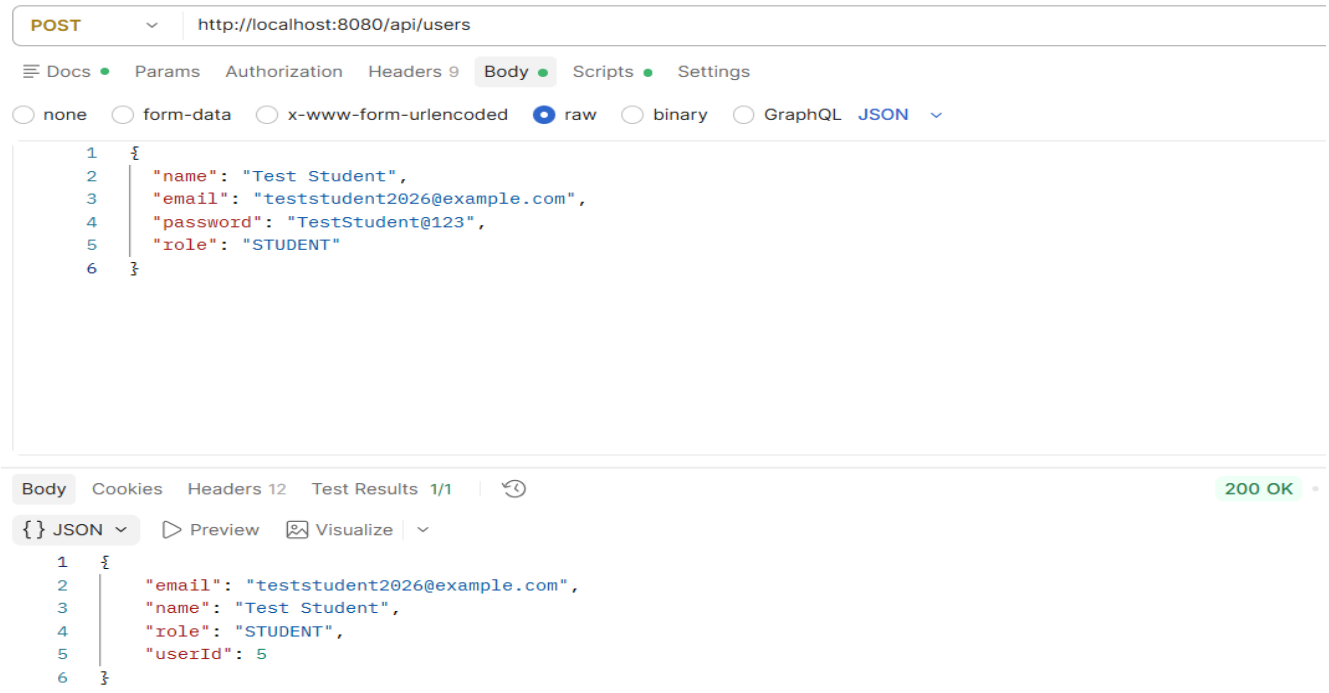


Figure 6: Successful execution of the user creation API using Postman.

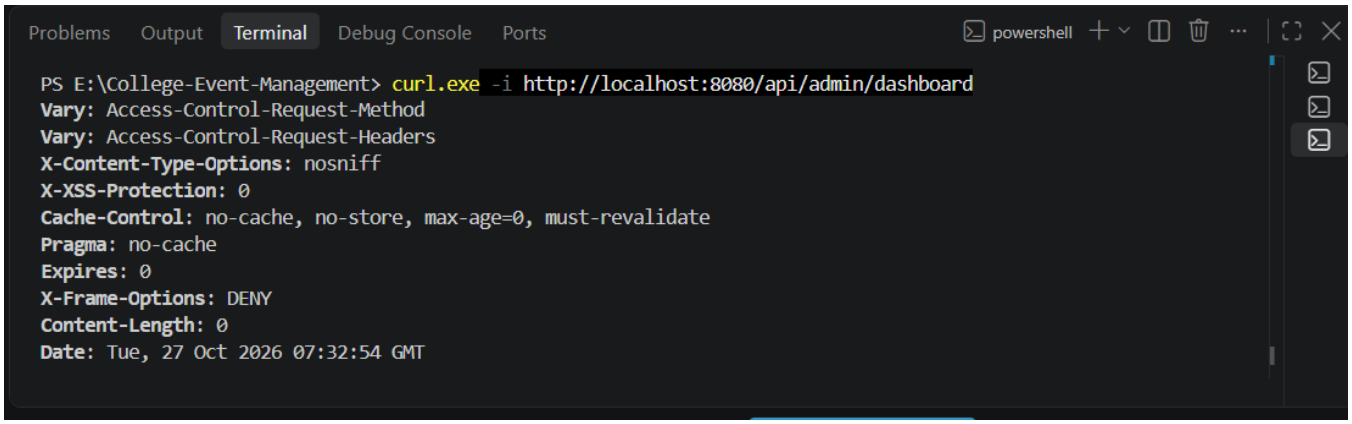


Figure 7: Verification of administrator access to the dashboard API using a valid authentication token, returning HTTP 200 OK.

```
PS E:\College-Event-Management> curl.exe -i -H "Authorization: Bearer YOUR_STUDENT_TOKEN" http://localhost:8080/
api/admin/dashboard
HTTP/1.1 403
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Tue, 27 Oct 2026 07:41:06 GMT
```

Figure 8: Verification of administrator access to the dashboard API using a valid authentication token, returning HTTP 200 OK.

```
PS E:\College-Event-Management> curl.exe -i -H "Authorization: Bearer YOUR_COORDINATOR_TOKEN" http://localhost:8
080/api/admin/dashboard
HTTP/1.1 403
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Tue, 27 Oct 2026 07:43:19 GMT
```

Figure 9: Verification of role-based access control by denying coordinator access to the administrator dashboard API, returning HTTP 403 Forbidden.

```
PS E:\College-Event-Management\backend> .\mvnw.cmd -Dtest=CollegeEventUnitTests#testPasswordEncryption test
[INFO] Scanning for projects...
[INFO]
[INFO] ------------< com.collegeevent:college-event-management >--------------
[INFO] Building college-event-management 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ college-event-management ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ college-event-management ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 37 source files with javac [debug parameters release 21] to target\classes
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ college-event-management ---
[INFO] skip non existing resourceDirectory E:\College-Event-Management\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ college-event-management ---
```

Figure 10: Successful execution of the unit test verifying BCrypt password encryption.

```
PS E:\College-Event-Management\backend> .\mvnw.cmd -Dtest=CollegeEventUnitTests#testJwtTokenGeneration test
[INFO] Scanning for projects...
[INFO]
[INFO] -------------< com.collegeevent:college-event-management >--------------
[INFO] Building college-event-management 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ college-event-management ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ college-event-management ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 37 source files with javac [debug parameters release 21] to target\classes
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ college-event-management ---
[INFO] skip non existing resourceDirectory E:\College-Event-Management\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ college-event-management ---
[INFO] Recompiling the module because of changed dependency.
```

Figure 11: Successful execution of the unit test verifying JWT token generation for a valid user.

```
--- resources:3.5.0:resources (default-resources) @ college-event-management ---
Copying 1 resource from src\main\resources to target\classes
Copying 0 resource from src\main\resources to target\classes

--- compiler:3.15.0:compile (default-compile) @ college-event-management ---
Recompiling the module because of changed source code.
Compiling 37 source files with javac [debug parameters release 21] to target\classes

--- resources:3.5.0:testResources (default-testResources) @ college-event-management ---
skip non existing resourceDirectory E:\College-Event-Management\backend\src\test\resources
```

Figure 12: Successful execution of the unit test verifying event entity creation with valid details.

```
PS E:\College-Event-Management\backend> .\mvnw.cmd -Dtest=CollegeEventUnitTests#testRegistrationCreation test
[INFO] Scanning for projects...
[INFO]
[INFO] -------------< com.collegeevent:college-event-management >--------------
[INFO] Building college-event-management 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ college-event-management ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ college-event-management ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 37 source files with javac [debug parameters release 21] to target\classes
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ college-event-management ---
[INFO] skip non existing resourceDirectory E:\College-Event-Management\backend\src\test\resources
[INFO]
```

Figure 13: Successful execution of the unit test verifying registration entity initialization.

```
E:\College-Event-Management\backend> .\mvnw.cmd -Dtest=CollegeEventUnitTests#testAttendanceStatusUpda
[INFO] Scanning for projects...
[INFO]
[INFO] -------------< com.collegeevent:college-event-management >--------------
[INFO] Building college-event-management 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ college-event-management ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
```

Figure 14: Successful execution of the unit test verifying attendance status update.

```
PS E:\College-Event-Management\backend> .\mvnw.cmd -Dtest=CollegeEventUnitTests#testCertificateCreation
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ college-event-management ---
[INFO] skip non existing resourceDirectory E:\College-Event-Management\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ college-event-management ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug parameters release 21] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.6:test (default-test) @ college-event-management ---
[INFO]
```

Figure 15: Successful execution of the unit test verifying certificate entity creation.

CORE IMPLEMENTATION

Module 1: User Interface / Presentation

The user interface was developed using React.js, with Bootstrap and CSS for styling. The application provides separate interfaces for Students, Coordinators, and Administrators. Students can browse events, register for events, and view their certificates. Coordinators can manage events, view participants, and mark attendance. Administrators can access the dashboard to monitor users, events, and registrations. The interface is designed to provide simple navigation and role-based access to the available features.

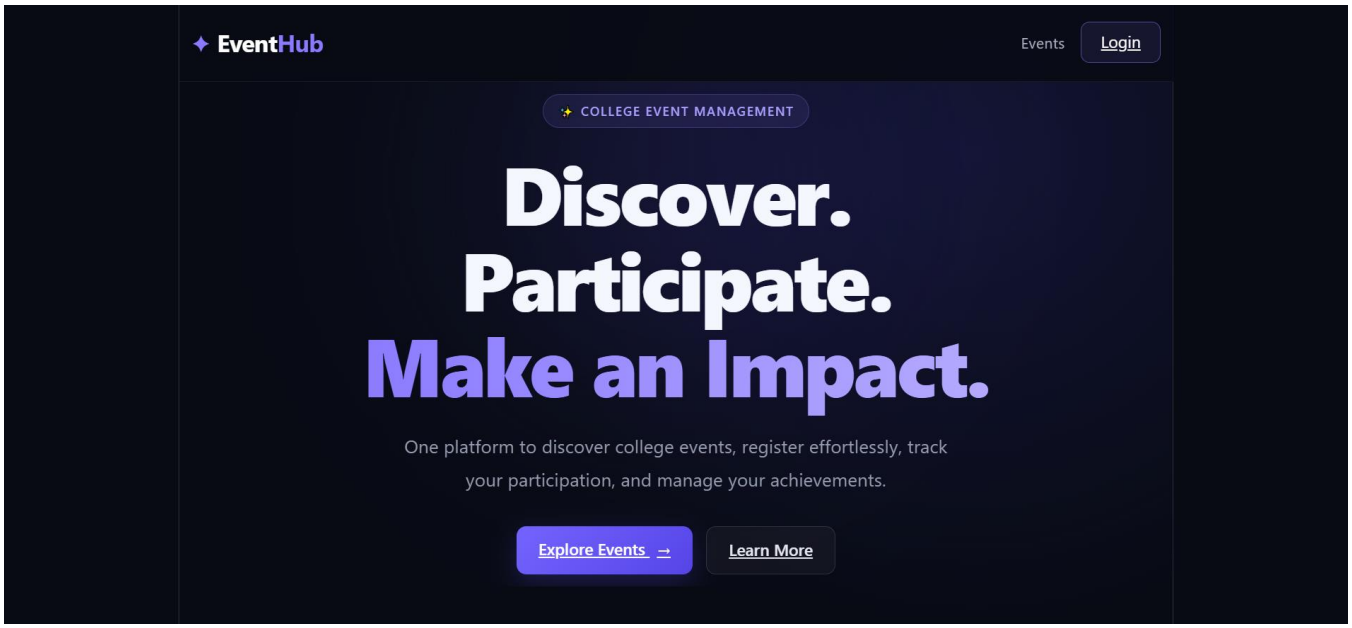


Figure 16: User Interface of the College Event Management & Participation System

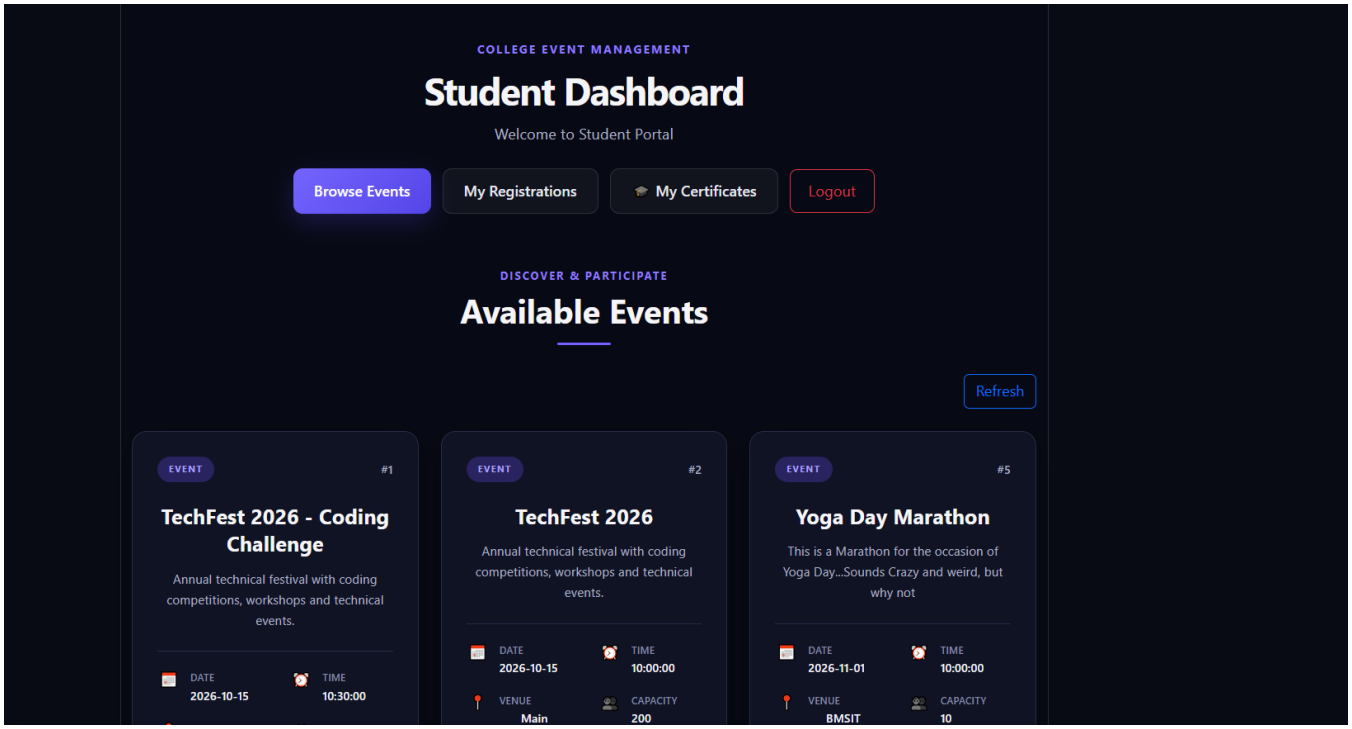


Figure 17: Student interface displaying available college events.

Module 2: Core Business Logic / Processing

The backend business logic was implemented using Java and Spring Boot. REST APIs handle user authentication, event management, event registration and cancellation, participant management, attendance tracking, and certificate generation. Spring Security and JSON Web Tokens (JWT) are used for authentication and role-based access control. The system verifies user permissions before allowing access to restricted administrative functions. Registration and attendance operations are processed through backend service classes to maintain consistent application behavior.

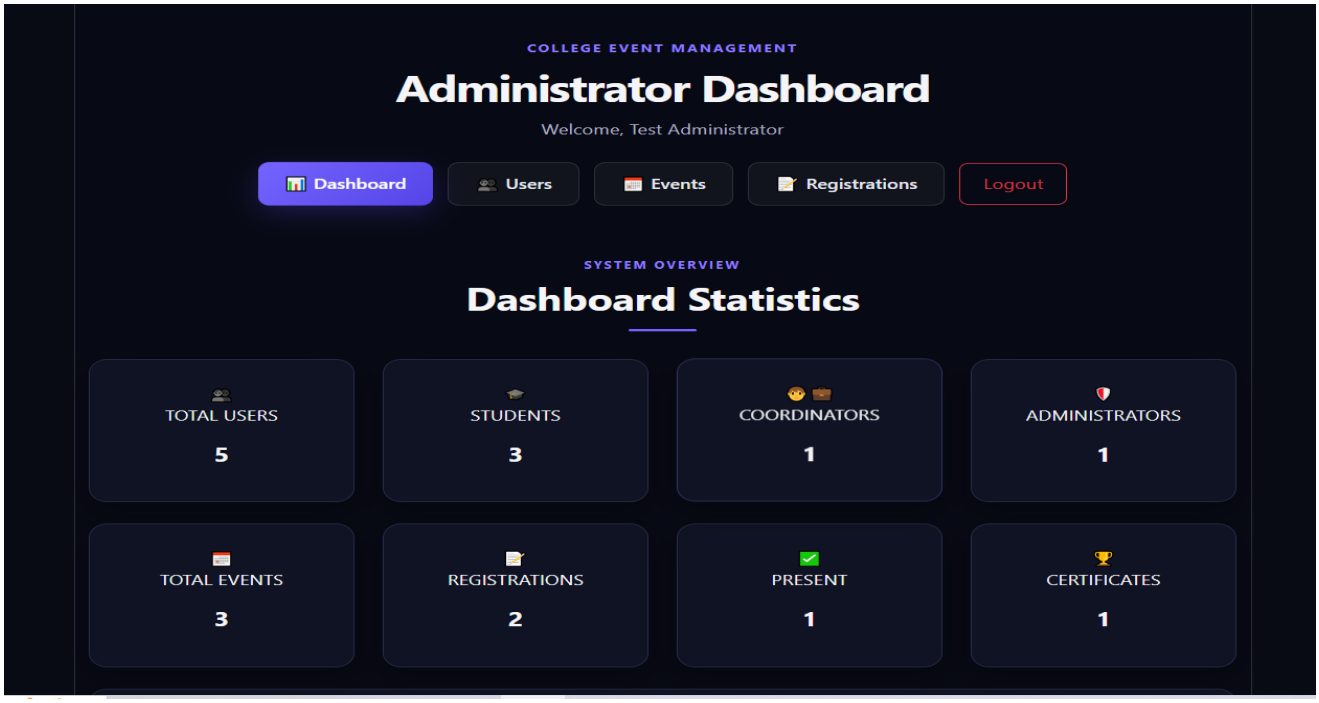


Figure 18: Administrator dashboard for monitoring users, events, and registrations.

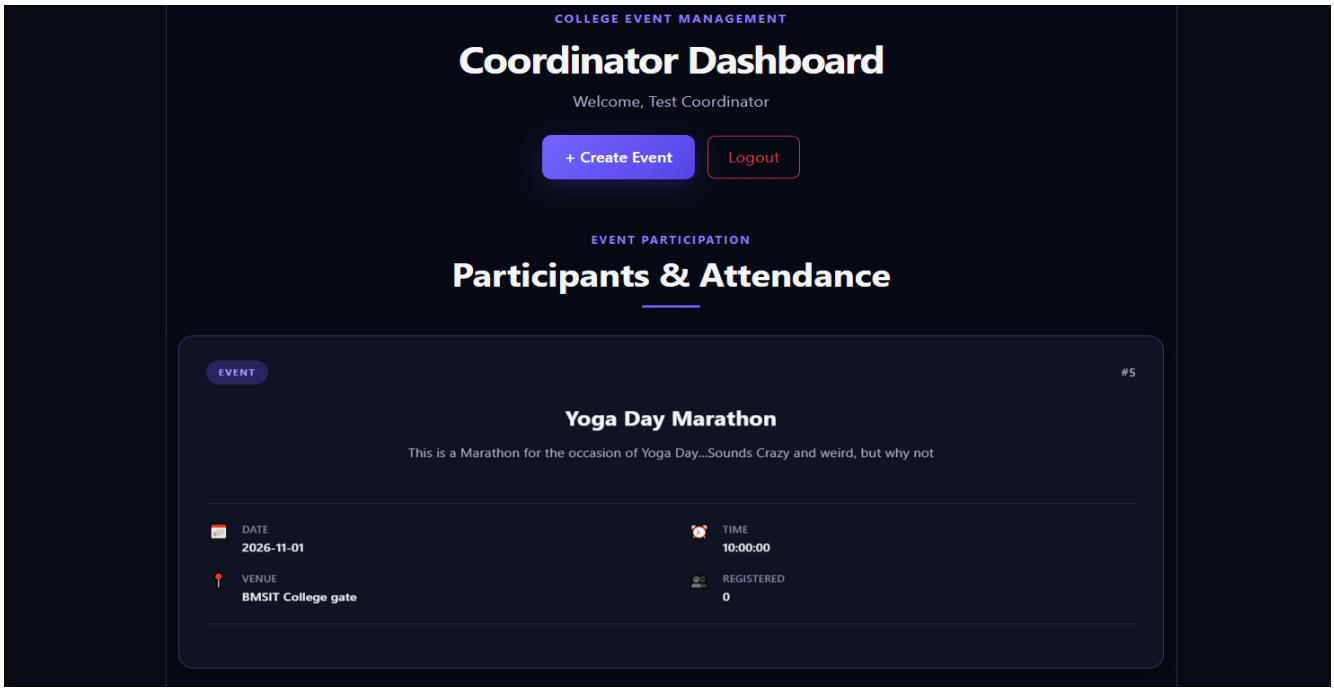


Figure 19: Event Participation and management functionality.

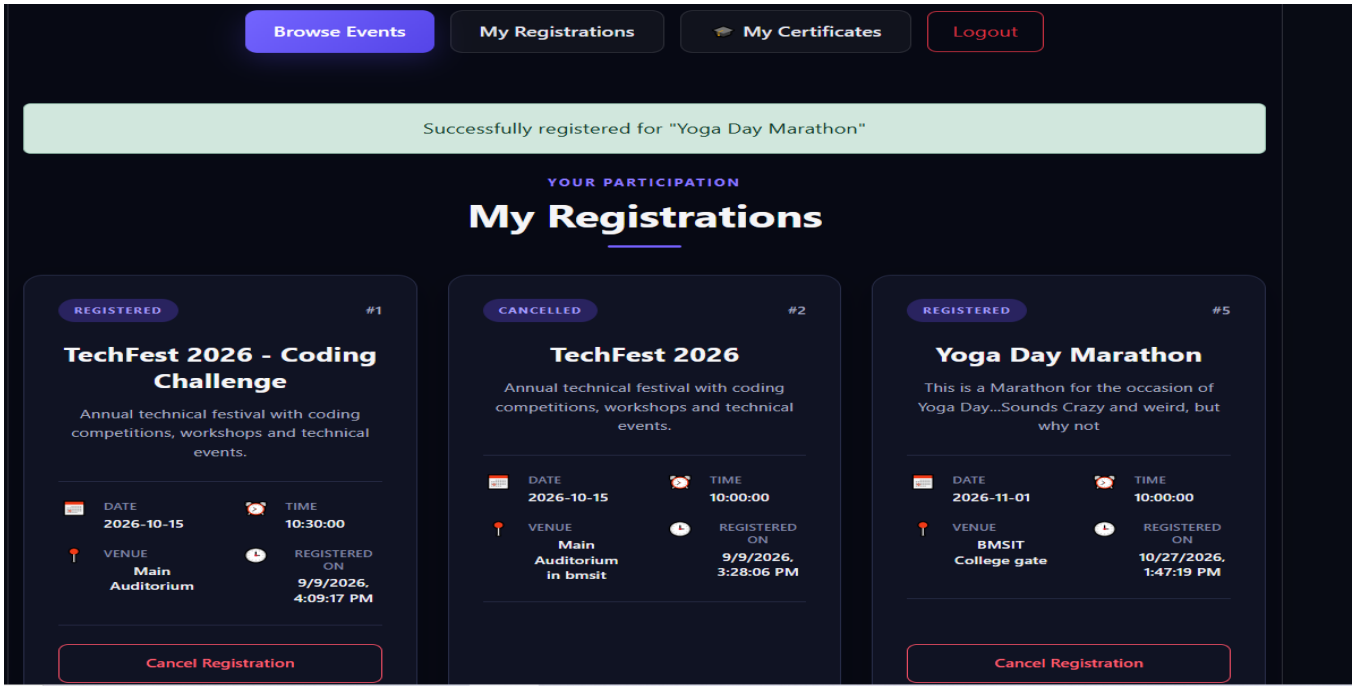


Figure 20: Event Registrations done by the Student

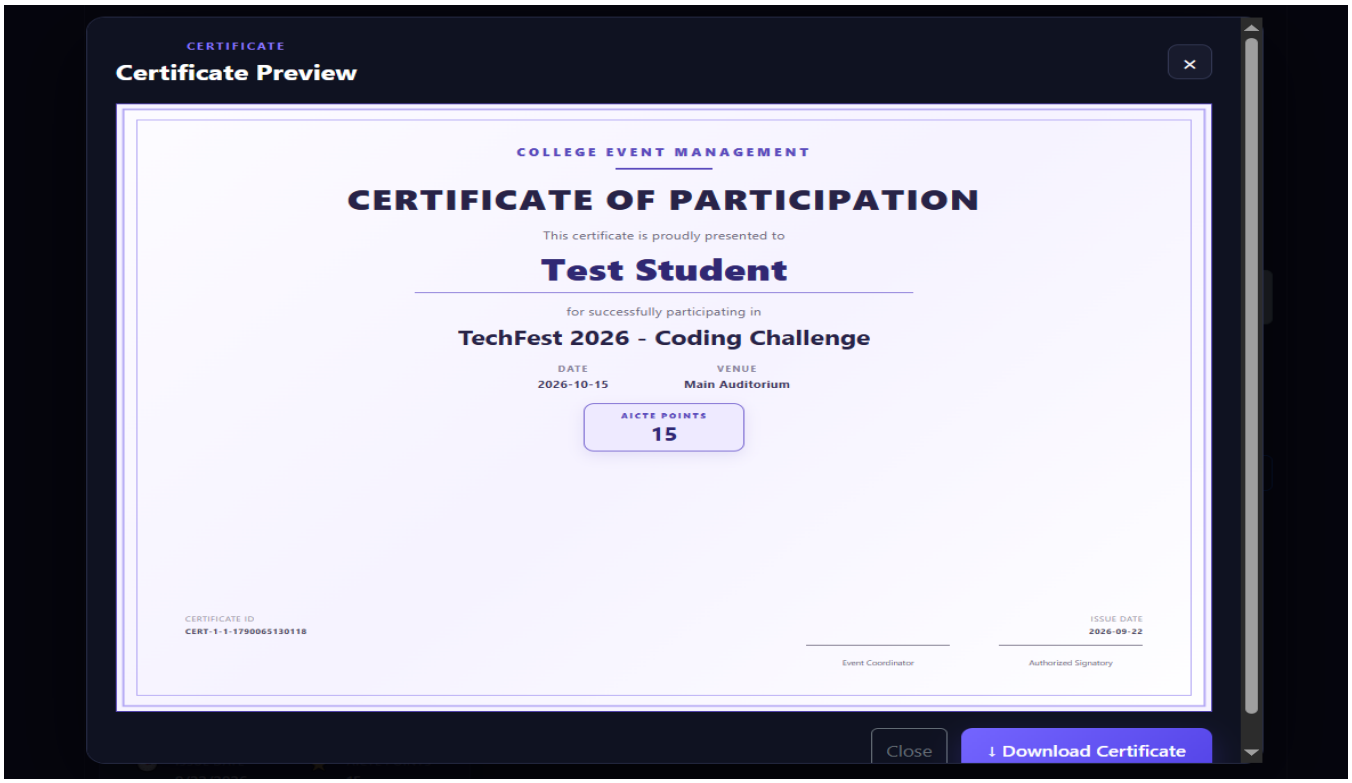


Figure 21: Certificate Preview and download

Module 3: Data Persistence and Integration

MySQL is used as the relational database for storing user accounts, event information, registrations, attendance records, and certificate-related data. Spring Data JPA and Hibernate manage database interactions through entity classes and repository interfaces. The React frontend communicates with the Spring Boot backend through REST APIs to submit requests and retrieve information. This integration allows application data to be stored, updated, and retrieved from the database as users perform different operations.

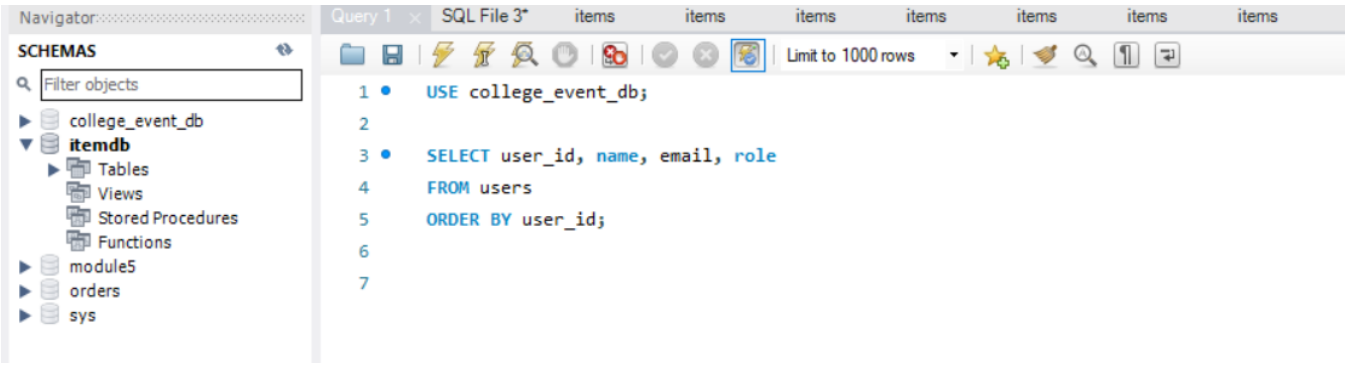


Figure 22: Data Base queries

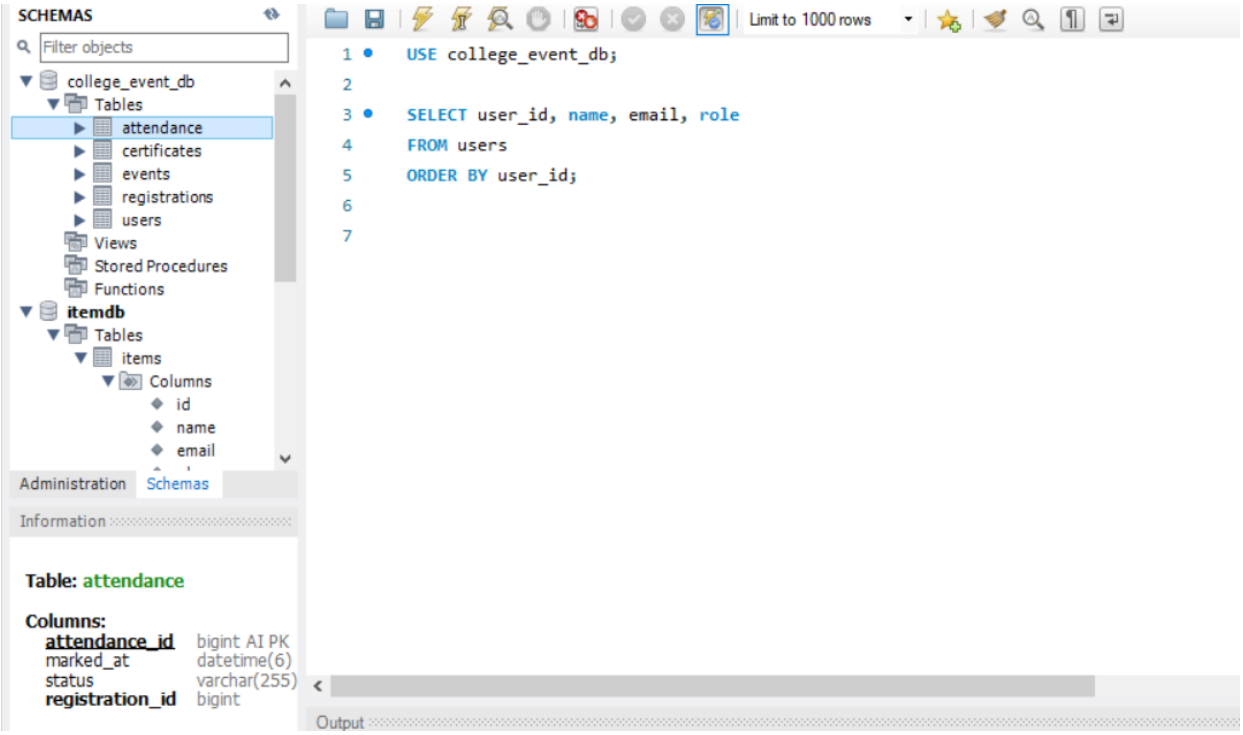


Figure 23: Tables Expansion in the Database

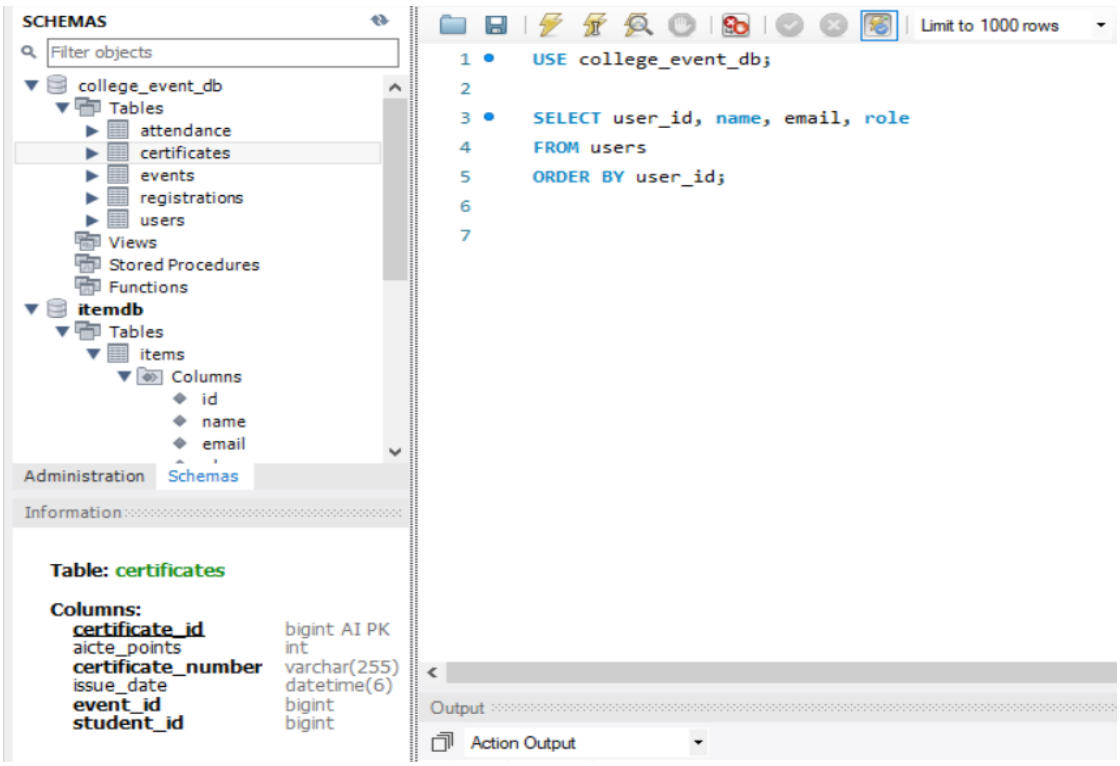


Figure 24: Function expansion of certificates

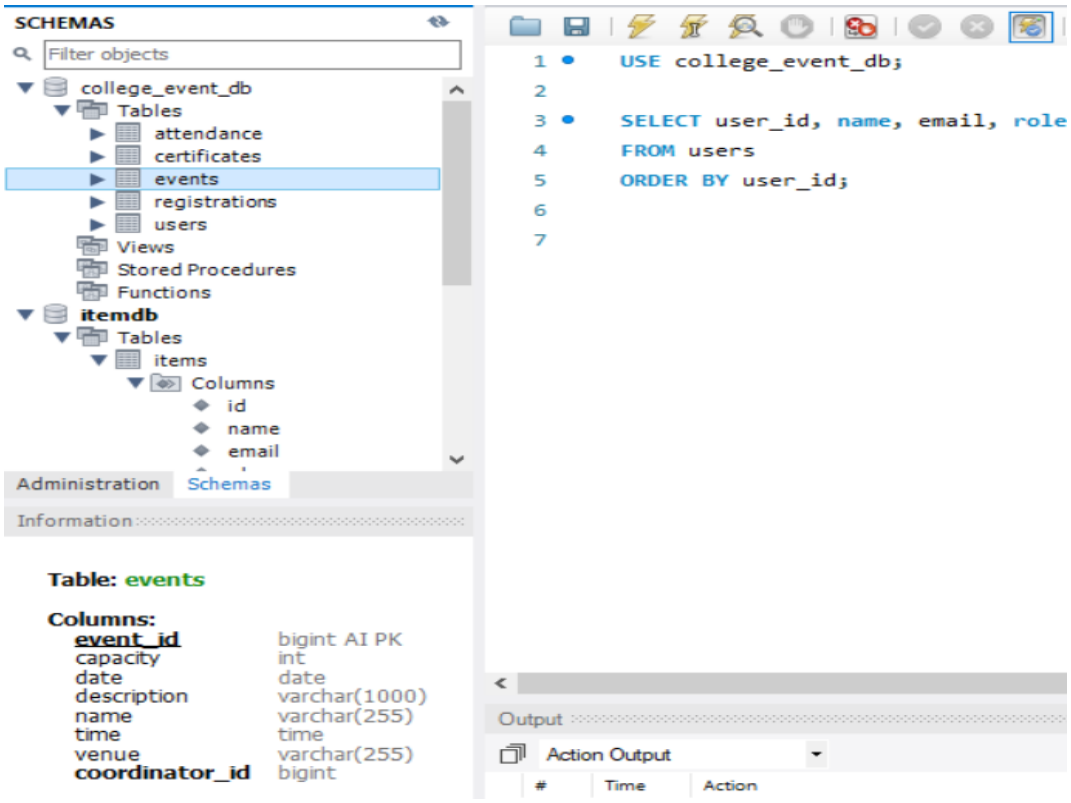


Figure 25: Function Expansion of Events.

EVALUATION RUBRICS

Construction Readiness, Testing and Implementation

(60 MARKS Scaled Down to 10 marks)

S.No	Evaluation Criteria	Excellent	Good	Needs Improvement	Max Marks	Marks Obtained
1.	Repo Setup & Code Structure	Well-organized GitHub/GitLab repo setup with modular folder organization. **(9-10 marks)**	Fairly organized repository with minor structural flaws. **(6-8 marks)**	Disorganized repo; lacks clear modular structure. **(0-5 marks)**	10	
2.	Static Code Analysis	Tool executed; zero critical bugs; report submitted. **(13-15 marks)**	Tool executed; minor code bugs present. **(9-12 marks)**	Static analysis not performed or results ignored. **(0-8 marks)**	15	
3.	Testing Execution	Working Unit and API/Integration tests with passing logs **(17-20 marks)**	Working unit tests, but limited integration test coverage. **(11-16 marks)**	Missing unit/integration tests or execution errors. **(0-10 marks)**	20	
4.	Implementation	Functional implementation of core modules matching architectural design. **(13-15 marks)**	Partial implementation with minor functional bugs or gaps **(9-12 marks)**	Incomplete or non-functional module implementation. **(0-8 marks)**	15	
				TOTAL MARKS		

Signature of Course Co-Ordinator

REFERENCES/BIBLIOGRAPHY

[1] React Team, “React Documentation.” [Online]. Available: https://react.dev/. [Accessed: Oct. 27, 2026].

[2] VMware, Inc., “Spring Boot Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-boot/documentation.html. [Accessed: Oct. 27, 2026].

[3] Oracle, “Java Documentation.” [Online]. Available: https://docs.oracle.com/en/java/. [Accessed: Oct. 27, 2026].

[4] Oracle Corporation, “MySQL Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/. [Accessed: Oct. 27, 2026].

[5] Spring, “Spring Security Reference.” [Online]. Available: https://docs.spring.io/spring-security/reference/. [Accessed: Oct. 27, 2026].

[6] L. Bradley and J. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force (IETF), May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519.html.

[7] Eclipse Foundation, “Eclipse Jakarta Persistence.” [Online]. Available: https://jakarta.ee/specifications/persistence/. [Accessed: Oct. 27, 2026].

[8] Hibernate Team, “Hibernate ORM Documentation.” [Online]. Available: https://hibernate.org/orm/documentation/. [Accessed: Oct. 27, 2026].

[9] Bootstrap Team, “Bootstrap Documentation.” [Online]. Available: https://getbootstrap.com/docs/. [Accessed: Oct. 27, 2026].

[10] Apache Software Foundation, “Apache Maven Project.” [Online]. Available: https://maven.apache.org/. [Accessed: Oct. 27, 2026].